

Федеральное государственное автономное образовательное
учреждение высшего образования
«Университет ИТМО»

Физико-технический мегафакультет

ОТЧЁТ

по учебному мини-проекту

Учебный алгоритм обнаружения опасного сближения двух космических аппаратов

Выполнил: студент группы L3316
Чернышов М.А.

Санкт-Петербург
2026

Содержание

1	Цель работы	2
2	Постановка задачи	2
3	Математическая модель движения	3
4	Алгоритм обнаружения опасного сближения	5
5	Общая схема работы алгоритма	6
6	Результаты моделирования	7
7	Ограничения модели	8
8	Вывод	9

1 Цель работы

Цель работы — собрать простой алгоритм, который по начальным координатам и скоростям двух космических аппаратов оценивает, насколько близко они подойдут друг к другу на заданном интервале времени.

Для этого в проекте нужно:

1. задать начальные состояния двух космических аппаратов
2. рассчитать их движение в центральном гравитационном поле Земли
3. получить траектории аппаратов
4. посчитать расстояние между ними во времени
5. найти минимальную дистанцию и момент максимального сближения
6. сравнить минимальную дистанцию с заданным порогом
7. проверить основные вычислительные части алгоритма тестами

2 Постановка задачи

Рассматриваются два космических аппарата, движущихся вокруг Земли. Для каждого аппарата известно начальное состояние, включающее радиус-вектор и вектор скорости:

$$\vec{y}_i(0) = \begin{bmatrix} x_i(0) \\ y_i(0) \\ z_i(0) \\ v_{x_i}(0) \\ v_{y_i}(0) \\ v_{z_i}(0) \end{bmatrix}, \quad i = 1, 2. \quad (1)$$

Здесь первые три компоненты задают положение аппарата в инерциальной системе координат, а последние три компоненты задают его скорость.

Необходимо определить, насколько близко аппараты подойдут друг к другу на заданном интервале времени:

$$t \in [t_0, t_f]. \quad (2)$$

Если минимальное расстояние между аппаратами оказывается меньше заданного порога безопасности, сближение считается потенциально опасным.

3 Математическая модель движения

В данной работе используется упрощённая модель движения космического аппарата в центральном гравитационном поле Земли. Земля считается телом, создающим центральное поле тяготения, а другие возмущения не учитываются.

Положение аппарата задаётся радиус-вектором:

$$\vec{r} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}. \quad (3)$$

Скорость аппарата задаётся вектором:

$$\vec{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}. \quad (4)$$

Полный вектор состояния имеет вид:

$$\vec{y} = \begin{bmatrix} \vec{r} \\ \vec{v} \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \end{bmatrix}. \quad (5)$$

Движение аппарата описывается системой дифференциальных уравнений:

$$\frac{d\vec{r}}{dt} = \vec{v}, \quad (6)$$

$$\frac{d\vec{v}}{dt} = \vec{a}. \quad (7)$$

Параметр μ называется гравитационным параметром Земли и равен произведению гравитационной постоянной G на массу Земли M :

$$\mu = GM.$$

Использование параметра μ удобно, поскольку в уравнениях орбитального движения величины G и M входят только в виде произведения. Для Земли в расчётах используется значение

$$\mu = 398600.4418 \text{ км}^3/\text{с}^2.$$

Именно этот параметр определяет величину гравитационного ускорения, действующего на космический аппарат. В центральном гравитационном поле Земли ускорение определяется выражением:

$$\vec{a} = -\mu \frac{\vec{r}}{|\vec{r}|^3}, \quad (8)$$

где μ — гравитационный параметр Земли:

$$\mu = 398600.4418 \text{ км}^3/\text{с}^2. \quad (9)$$

Модуль радиус-вектора равен:

$$|\vec{r}| = \sqrt{x^2 + y^2 + z^2}. \quad (10)$$

Тогда система уравнений движения в развернутом виде записывается как:

$$\begin{cases} \dot{x} = v_x, \\ \dot{y} = v_y, \\ \dot{z} = v_z, \\ \dot{v}_x = -\mu \frac{x}{(x^2 + y^2 + z^2)^{3/2}}, \\ \dot{v}_y = -\mu \frac{y}{(x^2 + y^2 + z^2)^{3/2}}, \\ \dot{v}_z = -\mu \frac{z}{(x^2 + y^2 + z^2)^{3/2}}. \end{cases} \quad (11)$$

Ускорение в этой модели не является постоянным: оно каждый раз пересчитывается по текущему положению аппарата относительно центра Земли.

Для построения траектории используется численное интегрирование системы ОДУ. В программе для этого применяется функция `solve_ivp` из библиотеки SciPy. Она получает начальное состояние аппарата и функцию правой части, а затем пошагово восстанавливает движение на заданном интервале времени.

Вектор состояния задаётся как

$$state = [x, y, z, v_x, v_y, v_z].$$

Функция правой части возвращает производные этого состояния:

$$\frac{dstate}{dt} = [v_x, v_y, v_z, a_x, a_y, a_z].$$

То есть координаты изменяются за счёт скорости, а скорость изменяется за счёт гравитационного ускорения.

4 Алгоритм обнаружения опасного сближения

После численного расчёта движения получаются две траектории:

$$\vec{r}_1(t), \quad \vec{r}_2(t). \quad (12)$$

Для каждого момента времени считается относительный вектор между аппаратами:

$$\Delta \vec{r}(t) = \vec{r}_1(t) - \vec{r}_2(t). \quad (13)$$

Расстояние между аппаратами равно длине этого вектора:

$$d(t) = |\Delta \vec{r}(t)|. \quad (14)$$

В координатной форме:

$$d(t) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}. \quad (15)$$

После этого задача сводится к поиску самого маленького значения среди всех рассчитанных расстояний.

5 Общая схема работы алгоритма

Алгоритм работы программы можно представить следующими шагами:

1. задать начальные координаты и скорости двух космических аппаратов;
2. задать временной интервал моделирования;
3. численно решить уравнения движения для первого аппарата;
4. численно решить уравнения движения для второго аппарата;
5. рассчитать расстояние между аппаратами для каждого момента времени;
6. найти минимальное расстояние d_{min} ;
7. определить момент максимального сближения t_{CA} ;
8. сравнить d_{min} с порогом безопасности d_{thr} ;
9. построить график зависимости расстояния от времени.

6 Результаты моделирования

В расчёте использовались следующие начальные состояния:

$$state_1 = \begin{bmatrix} 7008 & 0 & 0 & 0 & 7.546 & 0 \end{bmatrix},$$

$$state_2 = \begin{bmatrix} -7000 & 10 & 0 & 0 & 0 & 7.546 \end{bmatrix}.$$

Координаты заданы в километрах, скорости — в километрах в секунду. Интервал моделирования составил 7200 секунд.

По результатам расчёта были получены следующие значения:

Параметр	Значение
Минимальное расстояние	$d_{min} = 9910.42$ км
Момент максимального сближения	$t_{CA} = 4381.98$ с
Порог безопасности	$d_{thr} = 5$ км
Статус события	безопасное сближение

На рисунке 1 показано изменение расстояния между аппаратами во времени. Видно, что минимальная дистанция значительно больше выбранного порога безопасности.

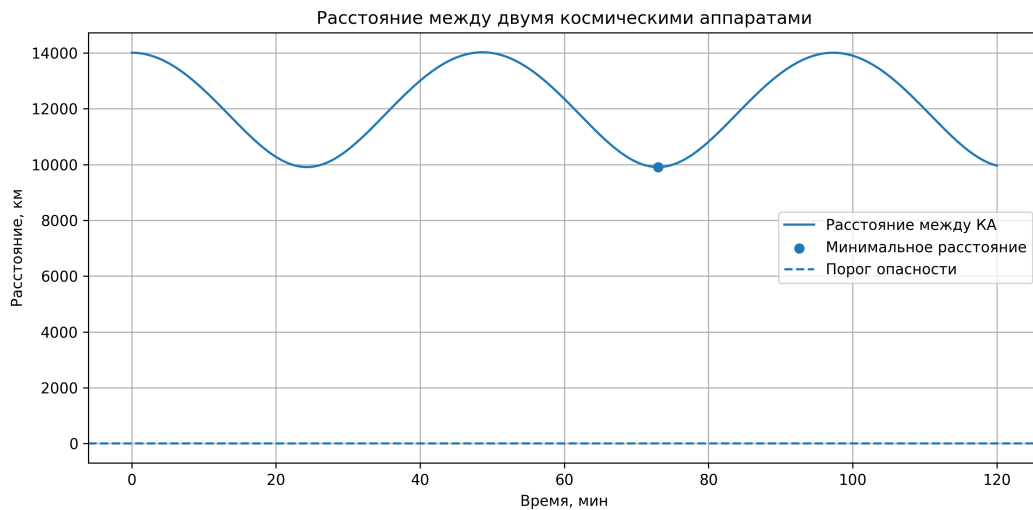


Рис. 1: Зависимость расстояния между космическими аппаратами от времени

На рисунке 2 показана пространственная визуализация рассчитанных траекторий.

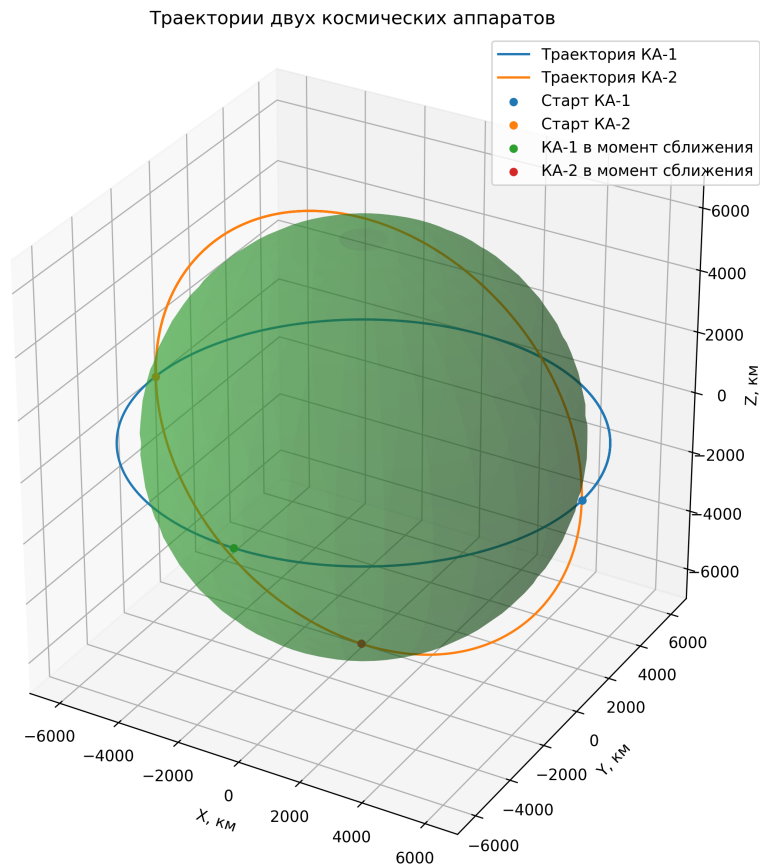


Рис. 2: 3D-визуализация траекторий двух космических аппаратов

7 Ограничения модели

Используемая модель является учебной и содержит следующие ограничения:

1. **Центральное поле.** Не учитываются J_2 и высшие гармоники.
2. **Атмосферное сопротивление.** Не учитывается торможение в LEO.
3. **Третьи тела.** Не учитывается влияние Луны и Солнца.
4. **Солнечное давление.** Не учитываются неконсервативные силы.
5. **Манёвры аппаратов.** Считается, что коррекции орбиты отсутствуют.
6. **Ошибки данных.** Начальные условия считаются точными.

8 Вывод

Я собрал простой, но наглядный инструмент: задал начальные состояния, промоделировал движение аппаратов в центральном поле Земли и посмотрел, как меняется расстояние между ними во времени. По минимуму дистанции и его времени легко определить, когда и насколько аппараты сблизятся, что и даёт базовую оценку потенциальной опасности.

Важно понимать, что это учебная модель — она не претендует на точное предсказание реальных столкновений. Я сознательно (ну и в силу того что это материал еще предстоит подтянуть) опустил сложные факторы (несферичность, атмосферу, притяжение Луны и Солнца, солнечное давление, возможные манёвры и погрешности начальных данных, неоднородность земли), поэтому численные значения d_{min} и t_{CA} нужно воспринимать как учебные, а не финальную экспертизу риска. Тем не менее алгоритм выполняет свою основную задачу: он позволяет быстро и прозрачно проверить идеи примерно — и служит хорошей отправной точкой для следующего шага, где можно добавить точности.